Exploring Bayesian Algorithms

Naive Bayes and TAN Bayes

Mitoceanu Ciprian-Minodor

November 2026

1 Introduction

In this paper, we will explore two Bayesian algorithms: Naive Bayes and Tree Augmented Naive Bayes (TAN).

I chose a tabular dataset to predict whether a song will go viral, based on musical features such as energy or danceability. The dataset can be explored at https://www.kaggle.com/datasets/vedauv/what-makes-a-song-go-viral-10000-tracks.

2 Data Preprocessing

Before performing the necessary calculations for the Bayesian algorithms, I cleaned the data and selected the columns that most influence the outcome, in order to avoid overfitting and reduce noise in the model. Therefore, I calculated the mutual information, and after running the program, I obtained the following results:

```
Mutual Information Scores (in bits):
----------------------------------------
streams             : 0.3060 bits
danceability        : 0.1212 bits
acousticness        : 0.1206 bits
energy              : 0.0854 bits
valence             : 0.0314 bits
instrumentalness    : 0.0041 bits
tempo_bpm           : 0.0016 bits
speechiness         : 0.0012 bits
duration_sec        : 0.0011 bits
liveness            : 0.0009 bits
loudness_db         : 0.0008 bits
release_year        : 0.0007 bits
```

The calculation clearly delineates the features that have the greatest influence on virality, highlighting a major difference between valence (0.314 bits) and instrumentalness, which drops to 0.0041 bits.

Additionally, I ignored the *Streams* column, as it is directly correlated with virality and would lead to data leakage, compromising the accuracy of the predictions.

3 Naive Bayes

For the naive algorithm, I started by calculating the *a priori* (prior) probability. For our dataset, approximately 80% of the songs are non-viral and 20% are viral. Next, I calculated the mean and variance for each feature.

Now that we have all the data calculated and stored, we can make predictions. Since our data is continuous, I used the Gaussian distribution to observe how a particular feature behaves depending on the target class (viral or non-viral).

To determine the class predicted by the model, I calculated the probability for each class by multiplying the *prior* probability by the probabilities of each feature.

$$\hat{y} = \arg\max_{y} \left(\log P(y) + \sum_{i=1}^{n} \log P(x_i \mid y) \right)$$

Note: Because we are working with extremely small probabilities, I applied the logarithm to avoid the *floating-point underflow* phenomenon. Thus, the multiplication operation was transformed into addition.

4 TAN Bayes

Although the naive algorithm is reliable, it assumes that each feature is independent of the others. In practice, this is rarely true.

To build the dependency tree between features, I calculated the conditional mutual information between each pair.

I constructed a complete weighted graph, where each edge has the conditional mutual information as its weight. From this, I extracted the maximum weight spanning tree using an algorithm like Kruskal or Prim. Subsequently, I randomly chose a node and directed the graph starting from it.

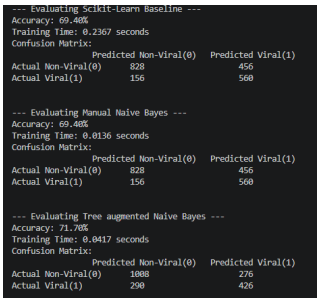
To calculate the probability of a song being viral, I proceeded similarly to the naive algorithm, but I also included the conditional probability of each feature given its parent.

The class prediction function becomes:

$$\text{Score}(c) = \log P(C = c) + \log P(x_{\text{root}} \mid C = c) + \sum_{i \neq \text{root}} \log P(x_i \mid \pi_{x_i}, C = c)$$

5 Results

To test the correctness of my implementation, I compared the obtained results with those of the Gaussian Naive Bayes algorithm from the *scikit-learn* library.



We can observe that the predictions of both naive algorithms are identical, but my proposed implementation runs twenty times faster. The TAN Bayes model achieves the best performance, although it provides only a marginal advantage over the naive variant.

6 Conclusion

In conclusion, the Bayesian approach proves to be an efficient solution for predicting song virality. While the Naive Bayes algorithm stands out for its excellent execution time, the TAN model captures the relationships between variables more accurately, offering a more robust theoretical model and a slight improvement in accuracy.